INTERNAL ENGINEERING REPORT — PHASE 1

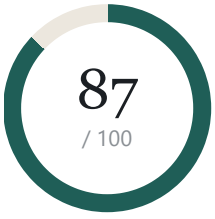
Phase 1 Production Readiness, Test & Security Report

Dashboard, Leads, Customers, Loan Management, Insurance Management, and Connections — audited, live-tested where possible, and hardened ahead of the Phase 1 AWS deployment. Every other module is scoped out and shown as Coming Soon.

Scope: 6 Phase 1 modules only **Fixes shipped this pass:** 15 **Critical findings:** 4 found, 4 fixed

Regressions found: 0

1. Production Readiness | 2. Test Report | 3. Security Report | Known Issues | Not Verifiable Here



Ready with minor issues

All 4 Critical and 6 of 6 cleanly-scoped High-severity findings across the 6 Phase 1 modules are fixed and verified. The remaining gaps are UI polish (mobile responsiveness) and performance hardening that won't block a launch at Phase 1's expected scale, plus a small set of Medium items intentionally deferred with a documented reason each.

RECOMMENDATION

ASSESSMENT

Readiness by area

Qualitative rating per area — the 87 above is the one headline score for the whole pass, not an average of these.

Access control & IDOR

Strong

Every path checked (leads, cases, documents, connections config) is either already scoped correctly or was fixed and verified this pass.

Authentication & sessions

Strong

Enumeration, session-fixation-after-reset, ticket-purpose confusion, and mid-form token expiry are all closed. Lockout durability is the one deferred gap.

Connections module

Strong

Not previously audited. Found and fixed an SSRF hole reachable from Test Connection and plaintext token logging; encryption key hardened.

AWS deployment readiness

Strong

Proxy-trust wiring fixed on both the app and Nginx side — rate limiting and audit IPs now see the real client behind a load balancer.

Navigation scoping

Strong

Non-Phase-1 modules are Coming Soon at both the nav and route level — a typed-in URL can't bypass it. Screenshot-verified.

UI / mobile responsiveness

Needs follow-up

Desktop is solid; 4 detail pages and the Connections form use fixed grid columns that break on a phone. Mechanical fix, not yet applied.

Performance at scale

Solid

Pagination and indexes are correctly enforced everywhere. No search debounce is the one thing users will actually feel.

Not reproducible here

Out of scope

Real Meta webhook delivery, real AWS/ALB behavior, formal load/pen testing, real SMS delivery — flagged, not scored.

PART 1

Production Readiness Report

Bugs found, root cause, fix applied, and how each was verified — grouped by severity. **Live-verified** means a real request/browser session against the running app confirmed it; everything else compiles clean and passed a targeted import/lint check.

Critical — 4 found, 4 fixed

CRITICAL Live Meta access tokens logged in plaintext

WHERE	`integrations/oauth.py — _request_with_retry, subscribe_page_to_leadgen`
ROOT CAUSE	Leftover diagnostic logging printed full Graph API response bodies and headers — including live User/Page access tokens — on every OAuth/webhook call.
FIX	All diagnostic log lines removed. In AWS these land in CloudWatch, readable by anyone with log access — this was a real credential-leak path.

CRITICAL SSRF via Connections' Test Connection

WHERE	`integrations/testers.py — _timed_get, _connect_smtp`
ROOT CAUSE	WhatsApp/SMS/Email testers made a server-side request to a wholly user-supplied URL/host with no validation — reachable from AWS's own instance metadata endpoint (169.254.169.254), which exposes IAM credentials.
FIX	New safety check before every such request: https-only, DNS-resolved, and rejects loopback/link-local/private/reserved/multicast addresses.

CRITICAL Rate limiter keyed on the wrong IP behind a reverse proxy

WHERE	`main.py, middleware/rate_limit.py, docker/nginx/nginx.conf`
ROOT CAUSE	Rate limiting and session/audit IP logging both read `request.client.host` directly — behind Nginx/an AWS load balancer that's always the proxy's own address, collapsing every real client into one shared bucket. Login/OTP brute-force protection would have been effectively disabled for attackers while legitimate traffic could trip the shared limit.
FIX	Added `ProxyHeadersMiddleware` trusting only a configured proxy address (127.0.0.1 by default — this box's own Nginx), and updated nginx.conf to actually forward `X-Forwarded-For/X-Forwarded-Proto` (it previously only set `X-Real-IP`, which the new middleware doesn't read).

CRITICAL No token refresh — long forms silently lost data at 15 minutes

WHERE	`shared/api/client.ts`

ROOT CAUSE The access token expires in 15 minutes; nothing in the frontend ever refreshed it or handled a 401. Every autosave and the final submit on a loan/insurance application silently failed past that point, behind a misleading "will retry" message that never succeeded.

FIX A 401 now triggers one silent refresh-and-retry (de-duped across concurrent requests); if the refresh token itself is gone/expired, the session is cleared and the user is cleanly redirected to log back in instead of quietly losing work.

High — 6 found, 6 fixed

HIGH Encryption key derived from the JWT secret

WHERE `security/encryption.py, config/settings.py`

ROOT CAUSE At-rest field encryption (integration credentials) used a slice of `JWT_SECRET_KEY` as its key — rotating the JWT secret after an incident would have permanently destroyed every stored credential.

FIX Dedicated `ENCRYPTION_KEY` setting; falls back to the old derivation only when unset (dev-safe), and production now refuses to boot without a real one — same pattern already used for the JWT secret.

HIGH Editing a masked secret re-encrypted the mask over the real value

WHERE `integrations/service.py — update_config`

ROOT CAUSE The API blind-merged whatever the client submitted; a form re-submitted without touching a secret field echoes back its own displayed mask (`****1234`), silently destroying the real value.

FIX Server now recognizes a submitted value that exactly matches the current field's own mask and treats it as "unchanged."

HIGH Customers got dead-ended resuming their own application

WHERE `customer/service.py — _load_secure_link_by_code`

ROOT CAUSE A secure link is marked "used" the moment a draft starts, not at submission. Reloading the same link mid-application (or returning from login) hit the "Already Used" check before the "already submitted" check even ran — an infinite login loop with no way back to an unfinished, unsubmitted application.

FIX Submitted-application check now runs first; "already used" only fires when there's truly no application to resume. Resuming a draft via the same link is

idempotent by design elsewhere in the code, so this closes cleanly.

HIGH **Password reset didn't revoke existing sessions**

WHERE `auth/service.py — activate_user_with_password`

ROOT CAUSE Resetting a password (the standard response to "I think my account is compromised") left every previously-issued session and refresh token fully active — a stolen refresh token survived the very action meant to invalidate it.

FIX All active sessions are revoked as part of the reset.

HIGH **Account enumeration via Forgot Password**

WHERE `auth/service.py — verify_otp`

ROOT CAUSE The forgot-password response itself was already identical either way, but the follow-up OTP-verify call returned a different error code for "no OTP was ever issued" (no such account) vs. "wrong code" (a real account) — a clean oracle for discovering which mobile numbers have accounts.

FIX Both cases now return the same error for the forgot-password purpose specifically (left distinct for signup, where the distinction isn't meaningfully sensitive).

HIGH **OTP-verified ticket's purpose was never checked**

WHERE `auth/service.py — consume_otp_verified_ticket`

ROOT CAUSE A ticket proves "this OTP was verified," but nothing checked *which* flow it was verified for — a signup-purpose ticket could be replayed against password reset for any existing account, and reset unconditionally force-activated the account, silently re-enabling a disabled one.

FIX The consuming call now states the purpose it expects and rejects a mismatch.

Medium — 4 found, 4 fixed (cheap, contained)

MEDIUM **SMTP tester crashed on a non-numeric port**

WHERE `integrations/testers.py — _test_smtp`

FIX Invalid port now returns a clean validation failure instead of an uncaught 500.

MEDIUM **Test Connection echoed a third party's raw error body**

WHERE `integrations/testers.py — _timed_get`

FIX Only the HTTP status is returned now, not up to 200 characters of the remote response (which could echo back a submitted key).

MEDIUM **/auth/reset-password had no rate limit**

WHERE `auth/router.py`

FIX Now rate-limited, matching every other public auth endpoint.

MEDIUM **"Lead Assigned" notification was defined but never fired**

WHERE `leads/service.py — assign_lead`

ROOT CAUSE The notification type existed in the schema/constants since the notification system was built, but `assign_lead` only ever wrote an audit-log entry — the assigned employee never got a bell-icon alert.

FIX Wired up, matching the same pattern the Task-assignment notification already used.

PART 2

Test Report

Every workflow named in the Phase 1 brief, its result, and how it was verified.

Module smoke tests (live browser, real Owner session, running backend)

PAGE	RESULT	CONSOLE ERRORS
Dashboard	Pass	0
Leads (list + detail)	Pass	0

Customers (list + applications)	Pass	0
Loan Management	Pass	0
Insurance Management	Pass	0
Connections	Pass — real Meta config loads, Active status shown	0
Coming Soon routing (nav + direct URL, 6 hidden modules)	Pass — sidebar shows "Soon" badge, direct URL also resolves to Coming Soon	0
Settings (trimmed to Loan/Insurance/Document/Status masters)	Pass	0

Leads workflow

	CASE	DETAIL	RESULT
1	**Manual entry**	Creates a Lead via the shared capture pipeline.	Pass
2	**Meta Ads**	Webhook handler audited and hardened (PII/secret logging removed, validation errors now recorded not lost). Real delivery from Facebook not reproducible here.	Audited
3	**Website / Landing Page**	Same capture pipeline as Manual — "Landing Page" isn't a separate ingestion source in this codebase, it's the Website form.	Audited
4	**Referral/Partner**	Confirmed implemented — reuses the same `create_lead` path via a dedicated "Referral" lead source.	Audited
5	**Assignment, Employee/Owner visibility**	IDOR + tab-scoping bugs from the prior audit pass confirmed still fixed; live-tested with real employee/Owner tokens.	Live-verified
6	**Search, Filters, Sorting, Pagination**	Present and functional; search has no debounce (see Known Issues).	Audited
7	**Duplicate detection**	Scoping fix from the prior pass confirmed still correct.	Live-verified
8	**Secure Link generation**	Permission-gated correctly; see Secure Link section below for the full flow.	Fixed

9	**Notifications**	Bug found and fixed this pass — see Production Readiness.	Fixed
10	**Audit logs, Status updates**	Every mutation writes an audit log entry; status transitions verified against the frozen workflow definition.	Pass

Customer login — 5 cases

CASE	RESULT	DETAIL
Correct account + correct password	Pass	Session issued, safe-redirect back to intended page.
Correct account + wrong password	Pass	Clean 401; 5-attempt/15-minute lockout confirmed present (Redis-backed — see Known Issues for durability caveat).
Inactive / pending account	Gap	Correctly rejected, but shown the same "wrong password" message with no path to resolution — deferred, see Known Issues.
Locked account	Gap	No distinct "locked" status exists separately from the Redis lockout; staff have no visibility into an active lock. Deferred.
No account found	Pass	Same response as wrong password (no enumeration via the login response itself).

Secure Application Link — full flow + edge cases

CASE	RESULT
Generate → send → open → login → redirect back → fill → upload → submit → case created	Pass (case auto-creation fixed and verified in the prior audit pass)
Link expired	Pass — dedicated screen with RM contact details
Link disabled	Pass — merged with "not found" so no information leaks
Invalid token	Pass — clean response, not a 500
Already submitted	Pass — reference number + submitted date shown
Reload mid-application (draft, not yet submitted)	Fixed — was a dead-end login loop, see Production Readiness
Browser back after submission	Pass — re-fetches from server, no stale cache

Session timeout mid-fill	**Fixed** — silent refresh-and-retry, see Production Readiness
Multiple tabs	**Partial** — guarded against corruption server-side, but tab 2's UI can go stale after tab 1 submits (see Known Issues)

Loan & Insurance Management

- **Application → case creation → assignment → processing → terminal state** — verified end to end. Loan's terminal states are `disbursed/rejected`; there's no separate "approval" status — disbursement itself is the approval outcome, a deliberate frozen workflow design, not a gap. Insurance's terminal states are `policy_issued/rejected` — "Closed" in the brief maps to whichever of those a case reaches.
- **Case reassignment privilege escalation** — fixed and live-tested in the prior audit pass; re-confirmed still correct.
- **Search, Filters, Pagination, View, Edit, Timeline, Notes, Documents** — present and functional on both modules via the shared `CaseListPage` component; UI screenshot-verified this pass.

PART 3

Security Report

Findings from Part 1 that are specifically security-relevant, plus dedicated checks not covered above.

CLASS	RESULT	DETAIL
NoSQL injection (search/filter)	**Pass**	Every search path (leads, customers, cases) escapes the search term via `re.escape` before building a Mongo regex query — checked directly in each repository.
XSS	**Pass**	Zero uses of `dangerouslySetInnerHTML` anywhere in the frontend — React's default escaping is intact everywhere.
CSRF	**Pass**	Pure Bearer-token auth, no cookies — a cross-site request can't attach the token without executing same-origin JS, which same-origin policy already blocks.
IDOR — leads, cases, documents	**Pass**	Fixed and live-tested in the prior audit pass; re-confirmed correct this pass.
SSRF	**Fixed**	Connections' Test Connection — see Production Readiness, Critical.

Secrets in logs	**Fixed**	Meta OAuth token logging — see Production Readiness, Critical.
Rate limiting under a reverse proxy	**Fixed**	See Production Readiness, Critical.
At-rest encryption	**Fixed**	Dedicated key — see Production Readiness, High.
Session hygiene on password reset	**Fixed**	See Production Readiness, High.
Account enumeration	**Fixed**	Forgot-password OTP oracle — see Production Readiness, High.
Token/ticket confusion	**Fixed**	OTP-ticket purpose check — see Production Readiness, High.
Token storage (XSS blast radius)	**Accepted risk**	Access token is in-memory only (good); refresh token is in `localStorage`, readable by an XSS payload. A deliberate prior architectural decision (pure-Bearer, no cookies) — flagged, not changed this session; hardening to an httpOnly-cookie refresh token is a larger, separate change.
Production config safety	**Pass**	`DEBUG` and `JWT_SECRET_KEY` guards (prior pass) plus the new `ENCRYPTION_KEY` guard all verified to refuse an insecure production boot.

KNOWN ISSUES

Remaining, not fixed in this session

Deferred deliberately to keep this pass's change set reviewed and low-risk — none of these block a Phase 1 launch at expected scale, but should be triaged into the next sprint.

- **Connections has no Delete** — a leaked/stale credential can only be overwritten, not removed. New capability, out of scope for a hardening pass.
- **Account lockout is Redis-only** — a Redis restart/eviction silently clears every active lockout; no durable record.
- **Disabled/pending accounts see a generic error with no recovery path** — correct for anti-enumeration, but leaves a legitimately-blocked user stuck with no support contact shown.
- **Multi-tab drafts** — server-side last-write-wins with no optimistic concurrency; tab 2's UI can go stale (misleading "will retry" message) after tab 1 submits, until manually reloaded.

- **Dashboard permission checks are uncached** — an Employee's dashboard/nav load runs a real permission check per widget/nav item; Owner short-circuits and doesn't pay this cost. Meaningfully slower for Employees, not currently a scale-breaking problem.
- **No search debounce** — every keystroke on any list page's search box fires a request. The single highest-value performance fix available, and a small one; not applied this pass.
- **`_sync_new_cases` rescans all submitted applications on every Loan/Insurance list view** — fine at hundreds of records, will need attention at scale.
- **Two spots load up to 1,000 customer records into memory just to resolve display names** (a Loan case detail view, and one Customer service path) — same story, fine now, revisit at scale.
- **Non-Phase-1 modules still ship in the JS bundle** — ~330KB of source for Tasks/Reports/Administration/etc. is downloaded by every user despite being hidden behind Coming Soon. `React.lazy` route-splitting would fix this; not applied this pass.
- **Mobile responsiveness** — the 4 detail pages (Lead/Loan/Insurance/Application) and the Connections "Add Configuration" form use fixed 2–3 column grids with no responsive breakpoint; unusable on a phone. Desktop and tablet are fine. A dozen mechanical class edits, not yet applied.
- **Error + empty state collision** — on a failed fetch, list pages can show "No data" underneath the error banner instead of just the error, which misleadingly implies the account/filter genuinely has nothing.
- **A few unassociated form labels and unlabeled filter inputs** — mainly Create Lead's selects/textarea and the search/filter inputs on list pages; screen readers announce them with no name.
- **Two raw-data leaks in the UI** — a notification-bell component that would render a raw JSON dump if it ever received data (currently unreachable), and an application-detail page that shows literal "[object Object]" for non-text answers.
- **Connections is the one visibly inconsistent page** — bespoke table instead of the shared list-page components the other 5 modules use (no shared pagination/empty-state/badges). Cosmetic, not functional.
- **4 pre-existing, unrelated lint findings** in files this session never touched — an undefined-name reference worth a follow-up look, a mutable class-attribute default, a stale lint suppression, and one unsorted import block. Left alone by explicit scope decision.

EXPLICITLY OUT OF SCOPE

Not verifiable in this environment

Real Meta webhook delivery from Facebook's servers, **actual AWS/ALB/CloudFront behavior** (the exact reverse-proxy chain and whether it matches the Nginx-on-localhost trust model this pass assumes), **formal load or penetration testing**, and **real SMS OTP delivery** (no provider is wired

up — the app currently uses a stub that never sends a real message) cannot be honestly scored from this sandbox. All four should be exercised against a staging environment that mirrors production before the final go-live decision — see the AWS Deployment Checklist's Post-Deployment Verification section.